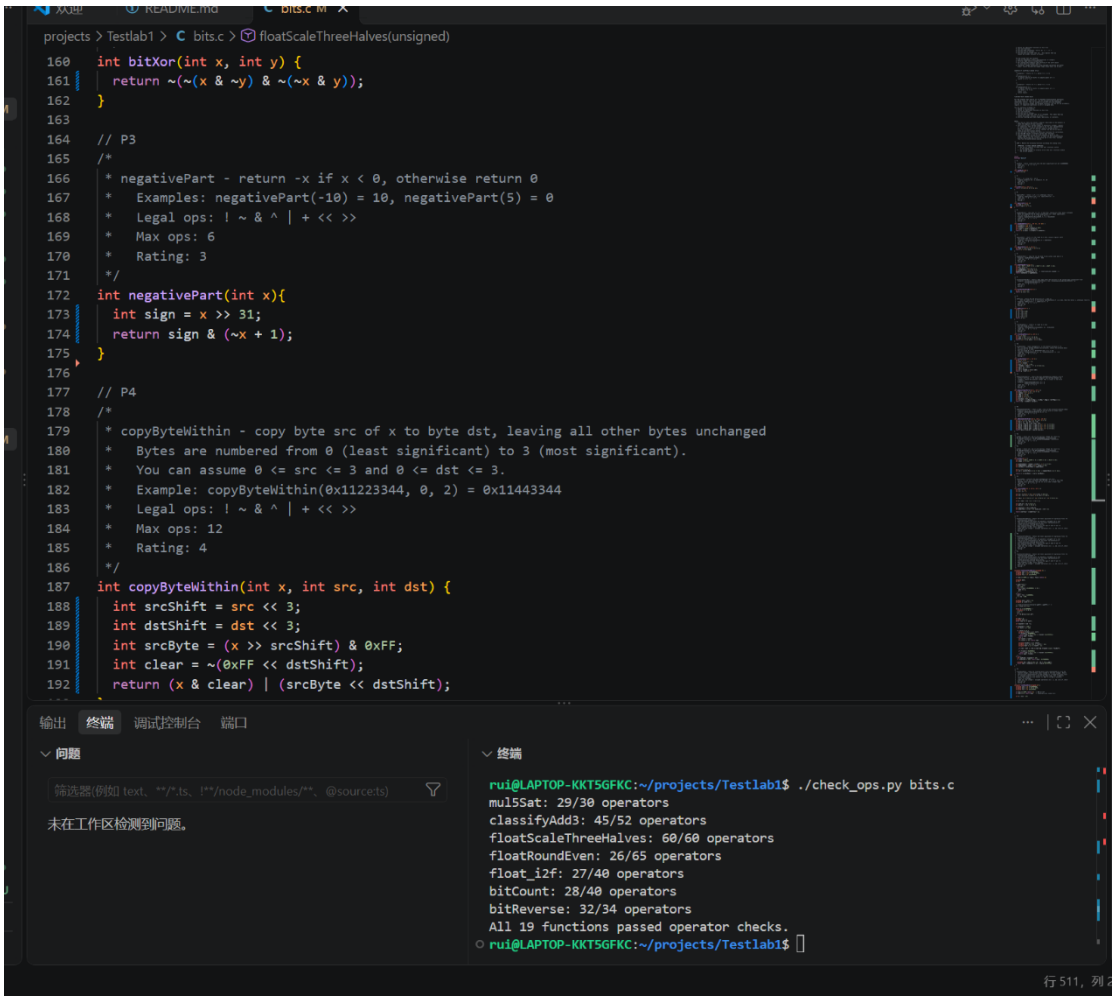
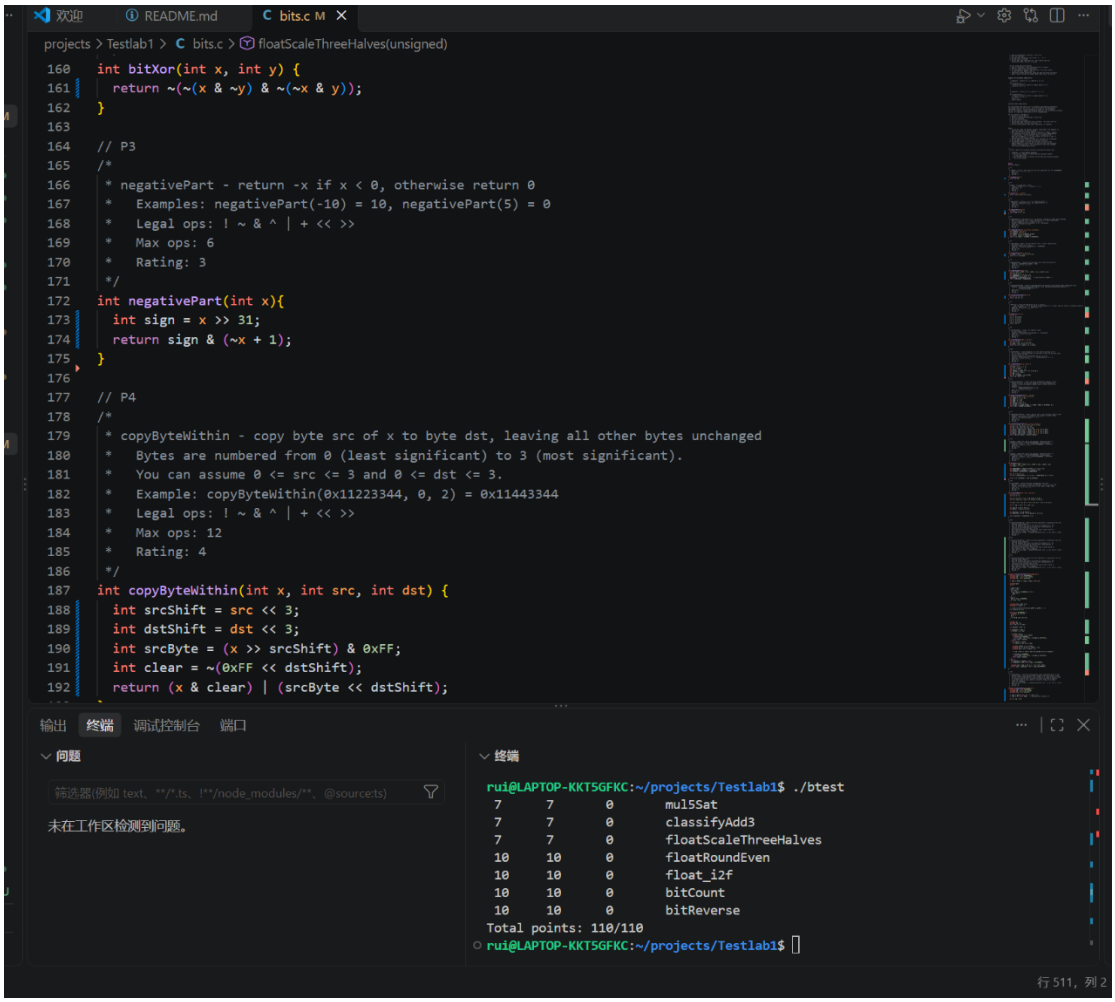
Lab1 实验报告

陈睿　　24300240174

在终端中执行 ./check_ops.py bits.c 后的截图



在终端执行./btest 后的截图



每个函数的思路

P1：思路：返回最高位为 1、其余位为 0 的掩码，即 0x80000000。直接把 1 左移 31 位即可。

P2：思路：

用 ~ 和 & 构造异或。异或的定义是“不同为 1”。

先分别找出：x & ~y：x 为 1、y 为 0 的位；~x & y：x 为 0、y 为 1 的位。

两者取反后再与非，等价于：$x⊕y$=~(~(x&~y)&~(~x&y))

P3:思路：思路：如果 x < 0，算术右移 31 位得到全 1，即 -1；如果 x >= 0，得到 0。

~x + 1 是 -x 的补码表示。用 sign 做掩码：负数时返回 -x，非负数时返回 0。

P4：思路：字节编号乘以 8 得到位移量。

先取出源字节 srcByte，再把目标字节位置清零，最后把源字节放到目标位置。

注意算术右移后要用 & 0xFF 只保留低 8 位。

P5 思路：机器执行的是算术右移，会在高位补符号位。

需要构造一个低 32-n 位为 1、高 n 位为 0 的掩码，把算术右移补出来的符号位清掉。

(1 << 31) >> n 会得到高 n 位为 1 的掩码，再左移 1 位并取反，就得到低 32-n 位为 1 的掩码。

P6 思路：每个字节内部交换高低 4 位。

lo 是每个字节低 4 位为 1 的掩码，hi 是每个字节高 4 位为 1 的掩码。

低 4 位左移 4 位，高 4 位右移 4 位。

因为右移是算术右移，所以右移后要用 & lo 清掉可能补出的符号位。

P7 思路：x + 1 会把最低的连续 1 变成 0，并把最低的 0 变成 1。

x | (x + 1) 会把最低的 0 位及其以下都变成 1。

因此 y 中最低的 0 位就是原 x 中第二个最低的 0 位。

~y & (y + 1) 可以孤立出 y 中最低的 0 位

P8 思路：

通过不断折半异或，把 32 位的奇偶性压缩到最低位。

最终 y & 1 是 x 中 1 的个数的奇偶性。

题目要求“1 的个数为偶数时返回 1”，所以取反后取最低位：~y & 1。

P9 思路：

循环右移 n 位，等价于：

右移 s = n & 31 位，并做逻辑右移；

左移 32 - s 位；

两者按位或。

mask 用来把算术右移补出的符号位清掉。

left 用补码方式计算 32 - s，再 & 31 防止移 32 位。

P10 思路：

把 x 分成商 q 和余数 r = x mod 2^n。

half = 2^(n-1)。

若 r > half，向上舍入；

若 r == half，看 q 的奇偶：奇数则进位，偶数则舍去；

若 r < half，舍去。

最后返回 (q + inc) << n。

P11 思路：

先计算不溢出的平均值：(x&y)+((x⊕y)>>1)

当 x + y 为奇数时，精确中点落在两个整数之间。

题目要求选择更靠近第一个参数 x 的整数。

因此判断 x 是否大于 y：

若 x > y，取 avg + 1；

否则取 avg。

sumOdd 表示和是否为奇数，xLarger 表示 x 是否更大。

P12 思路：

判断 x 是否在 a 和 b 之间，且端点顺序任意。

核心是构造有符号比较，避免减法溢出。

对于两个数 u 和 v，先看符号是否相同：

符号不同：非负者更大；

符号相同：用 u - v 的符号判断大小。

最后判断两种情况：

a <= x <= b

b <= x <= a

任意一种成立则返回 1。

P13 思路：

先计算 x * 5 = (x << 2) + x。

判断是否溢出：

INT_MAX / 5 = 0x19999999，即 limit；

若 x > limit，正溢出，应饱和到 INT_MAX；

若 x < -limit - 1，负溢出，应饱和到 INT_MIN。

用符号位和减法符号构造溢出掩码。

未溢出时返回 r，溢出时返回饱和值。

P14 思路：

先分两步计算 x + y 和 (x + y) + z，同时记录每一步是否发生有符号溢出。

c1、c2 分别表示两次加法的进位/溢出情况。

用 neg 统计三个操作数中负数的个数。

通过 neg - c1 - c2 判断真实和是否超出范围：

若结果为 0 但最终符号为负，说明正溢出，返回 1；

若结果为 1 但最终符号为正，或 m 更大，说明负溢出，返回 -1；

否则返回 0。

P15 思路：

计算 f * 3 / 2，输入输出都是单精度浮点的位模式。

主要步骤：

拆出符号 sign、阶码 exp、尾数 frac。

若为 NaN/Inf 或 0，直接返回原值。

把浮点数还原成“尾数 × 2^e”的形式：

规格化数：mant = frac | 0x800000，e = exp - 150；

非规格化数：mant = frac，e = -149，并左移直到最高位为 1。

计算 mant * 3，再除以 2，得到 q = (mant * 3) >> 1。

做 round-to-nearest-even。

若 q 超出 24 位，右移并调整指数。

根据最终指数判断：

若太小，输出非规格化数或 0；

若太大，输出无穷；

否则组装规格化浮点数。

核心是：把浮点运算转换成整数尾数运算，再重新编码。

P16 思路：

把浮点数舍入到最近的整数， halfway 情况舍入到偶数。

拆符号、阶码、尾数。

NaN/Inf 直接返回。

非规格化数或 0 舍入到 0，保留符号。

e = exp - 127

e < 0：数小于 1，按规则舍入到 0 或 1；

e >= 23：已经是整数，直接返回

否则取出有效尾数 full = frac | 0x800000

根据 shift = 23 - e 分出整数部分和小数部分。

小数部分与 half 比较：

大于一半：进位；

等于一半：看整数部分奇偶，奇数进位；

小于一半：舍去。

把整数部分重新编码成浮点数。

P17 思路：

把整数 x 转换成单精度浮点数位模式。

x == 0 返回 0。

取符号位 sign = x & 0x80000000。

取绝对值 abs。

找到最高位 1 的位置 msb。

指数域 exp = msb + 127。

若 msb <= 23，尾数直接左移对齐。

若 msb > 23，需要舍入：

右移 shift = msb - 23 位；

用 round-to-nearest-even 判断是否进位；

若尾数进位到 24 位，则尾数清零，指数加 1

返回 sign | (exp << 23) | frac。

P18 思路:

用经典的并行位计数法。

先构造掩码每两个位一组，统计 1 的个数;

每四个位一组，合并统计;

每八个位、十六个位、三十二个位依次合并;

最终得到整个 32 位中 1 的个数。

P19 每两个位一组，统计 1 的个数;

每四个位一组，合并统计;

每八个位、十六个位、三十二个位依次合并;

最终得到整个 32 位中 1 的个数。

先交换相邻 1 位;

再交换相邻 2 位;

再交换相邻 4 位;

再交换相邻 8 位

最后交换相邻 16 位。

每一步都把左右两半交换位置，最终整个 32 位完全反转。

对本实验的感受：本实验比较复杂，需要完成的函数较多，要求规则也较多，通过本实验能够增强我对代码编写，dug 调整等能力。